

TODO

Construir uma solução que:

▼ Consulte múltiplos provedores externos

O requisito **“consulte múltiplos provedores externos”** está implementado com dois adaptadores e um orquestrador que tenta os provedores em sequência.

Arquivos PHP

- `app/Infrastructure/Providers/Contracts/DebtProvider.php:7` define a porta comum que cada provedor implementa.
- `app/Infrastructure/Providers/ProviderA/ProviderAAdapter.php:10` consulta o endpoint configurado para o Provedor A, espera JSON e valida a estrutura básica da resposta.
- `app/Infrastructure/Providers/ProviderB/ProviderBAdapter.php:11` consulta o Provedor B, espera XML e transforma cada elemento `<debt>` em campos normalizados.
- `app/Application/Vehicle/GetVehicleDebts.php:23` coordena a consulta: tenta os provedores na ordem registrada, normaliza os dados para o modelo de domínio e usa o próximo provedor se houver falha. Se todos falharem, lança a condição tratada como HTTP 503.
- `app/Providers/AppServiceProvider.php:13` registra os dois adaptadores na aplicação.
- `config/services.php:4` lê as URLs dos provedores das variáveis `PROVIDER_A_URL` e `PROVIDER_B_URL`.
- `app/Domain/Debt/Debt.php:8` é o modelo canônico usado após a normalização.

Como testar no Postman

Na coleção `postman/vehicle-debt-api.postman/_collection.json`, estes requests cobrem a integração:

- **Search debts: TOTAL + payment simulations (Provider A JSON):** sucesso pelo Provedor A.

- **Provider fallback: A fails, B returns XML:** o mock faz A retornar 503; espera uma resposta bem-sucedida com `provider_b`, demonstrando o fallback e a normalização do XML.
- **All providers unavailable (503):** ambos os mocks falham; espera HTTP 503 e o código `providers_unavailable`.

Os fixtures que simulam os dois provedores estão em `tools/mock\_providers.py:17`. `ABC1234` usa A, `ZZZ0000` força o fallback para B, e `ZZZ9999` simula a indisponibilidade dos dois.

▼ Normalize os dados em um modelo canônico

A normalização fica entre os adapters e o restante da aplicação: cada adapter lê o formato específico do provedor e entrega os campos `id`, `type`, `amount` e `due_date`. O caso de uso valida esses campos e os converte para o modelo canônico `Debt`; juros, filtros e pagamentos depois trabalham só com esse modelo.

- **Provedor A:**
`app/Infrastructure/Providers/ProviderA/ProviderAAdapter.php:14` lê o JSON e retorna `debts`.
- **Provedor B:**
`app/Infrastructure/Providers/ProviderB/ProviderBAdapter.php:21` percorre os elementos XML e produz os mesmos campos que o adapter A.
- **Normalização e validação:**
`app/Application/Vehicle/GetVehicleDebts.php:61` converte `type` em `DebtType`, `amount` em `Money` (centavos inteiros) e `due_date` em `DateTimeImmutable`. Os campos resultam em um `Debt` (`app/Domain/Debt/Debt.php:8`), que usa o enum `DebtType` (`app/Domain/Debt/DebtType.php:5`).

Por exemplo, estes dados distintos:

```
{"id":"ipva-1","type":"IPVA","amount":"1200.00","due_date":"2026-09-01"}
```

```
<debt><id>ipva-1</id><type>IPVA</type><amount>1200.00</amount><due_date>2026-09-01</due_date></debt>
```

acabam representados pelo mesmo objeto `Debt`. O fluxo de seleção e fallback dos providers está em `app/Application/Vehicle/GetVehicleDebts.php:27`.

Cobertura no Postman

Na coleção Vehicle Debt API (postman/vehicle-debt-api.postman\collection.json):

- **Search debts: TOTAL + payment simulations (Provider A JSON)** verifica a resposta canônica com dados do JSON.
- **Provider fallback: A fails, B returns XML** verifica `provider_b` e campos canônicos, como `original_amount` e `payment_options`, após receber XML.
- **SOMENTE_IPVA** e **SOMENTE_MULTA** verificam que o filtro usa os tipos normalizados.
- **Zero-debt vehicle** cobre XML/JSON vazio sem produzir dívidas.

O mock local em `tools/mock\_providers.py:17` fornece os exemplos JSON e XML usados por esses requests.

▼ Calcule valores atualizados com juros (juros simples)

Os juros são calculados como **juros simples**: $\text{principal} \times \text{taxa diária} \times \text{dias}$ de atraso. Os dias são limitados a zero para dívidas ainda não vencidas.

Implementação PHP

- `app/Domain/Debt/DebtCalculator.php:13` escolhe a regra pelo tipo da dívida.
- `app/Domain/Debt/Interest/IpvaInterestRule.php:11` calcula $\text{principal} \times 0,33\% \times \text{dias}$ e limita o resultado a 20% do principal.
- `app/Domain/Debt/Interest/MultaInterestRule.php:11` calcula $\text{principal} \times 1\% \times \text{dias}$, sem teto.
- `app/Domain/Shared/Money.php:31` faz as operações em centavos inteiros e arredonda `HALF\_UP`.
- `app/Application/Vehicle/GetVehicleDebts.php:48` calcula os juros e soma ao principal para formar `total`.

Exemplo: uma dívida IPVA de `100.00`, vencida há 10 dias, gera `3.30` de juros e total `103.30`. Para MULTA de `100.00` atrasada há 40 dias, são `40.00` de

juros, sem teto.

Testes

Em `tests/Unit/MoneyAndDebtTest.php:21`, há testes para o cálculo IPVA antes do teto, o teto de 20%, MULTA sem teto e dívida ainda não vencida. Também corriji a expectativa do teste do teto: com principal de `1,000.00`, 20% corresponde a `200.00`.

No Postman, **Search debts: TOTAL + payment simulations (Provider A JSON)** confere os juros de cada fixture usando `days_overdue`, o principal e a regra correspondente ao tipo. Também verifica que `total` é principal mais juros. Os dados têm datas relativas: incluem débitos vencidos e futuros.

A coleção JSON foi validada, mas não executei o PHPUnit neste passo.

▼ Simule formas de pagamento (PIX e cartão de crédito)

A simulação foi implementada com estratégias separadas para PIX e cartão. As opções aparecem no JSON da consulta, tanto para cada dívida quanto para o total; **não existe endpoint que efetue pagamentos.**

Implementação PHP

- `app/Domain/Payment/Strategies/PixPayment.php:8` aplica 5% de desconto.
- `app/Domain/Payment/Strategies/CreditCardPayment.php:8` oferece 1x sem juros e calcula 6x e 12x pelo sistema Price/PMT, com taxa mensal de 2,5%.
- `app/Domain/Payment/PaymentSimulator.php:9` combina as estratégias.
- `app/Domain/Payment/PaymentOption.php:7` representa método, parcelas, total e valor da parcela.
- `app/Application/Vehicle/GetVehicleDebts.php:48` inclui as opções na resposta da consulta.

Exemplo para `R$ 1.000,00`:

Opção	Parcela	Total
PIX	R\$ 950,00	R\$ 950,00
Cartão 1x	R\$ 1.000,00	R\$ 1.000,00
Cartão 6x	R\$ 181,56	R\$ 1.089,36

Opção	Parcela	Total
Cartão 12x	R\$ 97,49	R\$ 1.169,88

O cartão parcelado usa a fórmula $PMT = P \times i / (1 - (1 + i)^{-n})$. Os cálculos são feitos com centavos inteiros, sem floats.

Cobertura dos testes

No Postman, **Search debts: TOTAL + payment simulations (Provider A JSON)** verifica o desconto do PIX, a presença de cartão 1x/6x/12x, o cálculo Price das parcelas e se o total parcelado corresponde ao valor da parcela vezes o número de parcelas.

Em tests/Unit/MoneyAndDebtTest.php:49, o teste unitário compara os valores calculados para R\$ 1.000,00. Corrigi também a expectativa anterior do PIX para R\$ 950,00. A coleção Postman foi validada como JSON; não executei PHPUnit nesta etapa.

▼ Permita pagamento total ou parcial (por tipo de débito)

O requisito é atendido pela seleção dos débitos **por tipo** antes de calcular o subtotal e simular as formas de pagamento:

- Sem informar `debt_type`, ou usando `TOTAL`, a consulta inclui todos os débitos.
- `SOMENTE_IPVA` inclui apenas débitos de IPVA.
- `SOMENTE_MULTA` inclui apenas multas.

O controller valida o filtro em `app/Http/Controllers/VehicleDebtController.php:27`. O caso de uso filtra os débitos antes de somar seus valores e gerar as opções de pagamento em `app/Application/Vehicle/GetVehicleDebts.php:41`.

Exemplo de request:

```
POST /api/vehicle-debts
Content-Type: application/json

{"plate": "ABC1234", "debt_type": "SOMENTE_IPVA"}
```

A resposta contém apenas débitos IPVA, o `total` desse subconjunto e as opções PIX/cartão calculadas sobre esse total. `TOTAL` segue o mesmo fluxo

sem aplicar o filtro.

Cobertura no Postman

Na coleção Vehicle Debt API (postman/vehicle-debt-api.postman_collection.json), os requests **Search debts: SOMENTE_IPVA** e **Search debts: SOMENTE_MULTA** verificam que só aparecem débitos do tipo solicitado. **Search debts: TOTAL + payment simulations** cobre a seleção total e suas formas de pagamento.

O escopo parcial aqui é por categoria: a API não permite escolher débitos individuais dentro do mesmo tipo nem efetua uma transação; ela simula opções para o subtotal selecionado.

▼ Seja extensível para novos provedores e novos tipos de débito

A extensão é feita por contrato, mantendo o orquestrador independente dos formatos e nomes dos provedores. Há pontos explícitos de registro e de adição de regras para novos tipos.

Adicionar um provedor

app/Infrastructure/Providers/Contracts/DebtProvider.php:7 define a interface que o novo adapter implementa. Ele busca os dados no formato do provedor e os entrega nos campos canônicos `id`, `type`, `amount` e `due_date`.

```
final class ProviderCAAdapter implements DebtProvider
{
    public function name(): string
    {
        return 'provider_c';
    }

    public function debts(Plate $plate): array
    {
        // Consultar o endpoint C e converter a resposta para os campos canônicos.
    }
}
```

O fluxo de fallback em app/Application/Vehicle/GetVehicleDebts.php:27 consome qualquer adapter registrado. Para incluir o C na sequência, registre `ProviderCAAdapter::class` na lista em app/Providers/AppServiceProvider.php:15. O registro é explícito; não há descoberta automática de plugins.

Adicionar um tipo de débito

O modelo aceita tipos definidos em `app/Domain/Debt/DebtType.php:5`. Para adicionar, por exemplo, `LICENCIAMENTO`, seria necessário:

1. Adicionar `case LICENCIAMENTO = 'LICENCIAMENTO'` ao enum.
2. Criar a regra de juros que implementa `app/Domain/Debt/Interest/InterestRule.php:9`.
3. Incluir essa regra no `match` de `app/Domain/Debt/DebtCalculator.php:13`.

A normalização converte o tipo recebido para o enum; após adicioná-lo, o filtro `SOMENTE_LICENCIAMENTO` também passa pela validação existente.

O que o Postman cobre hoje

Na coleção Postman (`postman/vehicle-debt-api.postman_collection.json`):

- **Search debts: TOTAL + payment simulations (Provider A JSON)** verifica o caminho do Provider A.
- **Provider fallback: A fails, B returns XML** verifica o adapter B, o fallback e os campos normalizados.
- **SOMENTE_IPVA** e **SOMENTE_MULTA** cobrem os tipos atualmente implementados.
- **Unknown debt type (422)** confirma que tipos ainda não cadastrados são rejeitados.

Um request para o novo tipo seria:

```
{"plate": "ABC1234", "debt_type": "SOMENTE_LICENCIAMENTO"}
```

Ele só deve ser adicionado à coleção como teste de sucesso depois de implementar o enum/regra e incluir esse débito no mock provider. Hoje, o Postman cobre a extensão entre os dois adapters existentes e a rejeição de tipos desconhecidos, mas ainda não testa um terceiro provedor ou um terceiro tipo.

▼ Seja resiliente a falhas (fallback entre provedores)

A resiliência combina tentativas HTTP limitadas com fallback sequencial: o sistema tenta o Provider A e, se a chamada ou o payload falhar, tenta o Provider B. Se ambos falharem, retorna HTTP 503.

Implementação PHP

Em `app/Application/Vehicle/GetVehicleDebts.php:27`, o orquestrador percorre os adapters. Falhas são registradas com a placa mascarada, e a consulta continua para o próximo provedor. Se nenhum fornecer dados válidos, o caso de uso indica indisponibilidade.

Os adapters aplicam timeout e tentativas limitadas; por exemplo, `app/Infrastructure/Providers/ProviderA/ProviderAAdapter.php:18` usa `timeout(...)` e `retry(2, 150)`. O controller converte a falha de todos os provedores em HTTP 503 em `app/Http/Controllers/VehicleDebtController.php:39`.

O circuito breaker mencionado como desejável **não foi implementado**; a proteção atual é timeout, retry e fallback.

Testes

Na coleção Postman:

- **Provider fallback: A fails, B returns XML** usa a placa `ZZZ0000`: A falha, B responde, e o teste espera HTTP 200 com `provider_b`.
- **All providers unavailable (503)** usa `ZZZ9999`: os dois mocks falham, e o teste espera HTTP 503 com `providers_unavailable`.

Os mesmos fluxos têm testes de integração em `tests/Feature/VehicleDebtApiTest.php:22`, incluindo fallback para XML e indisponibilidade total.